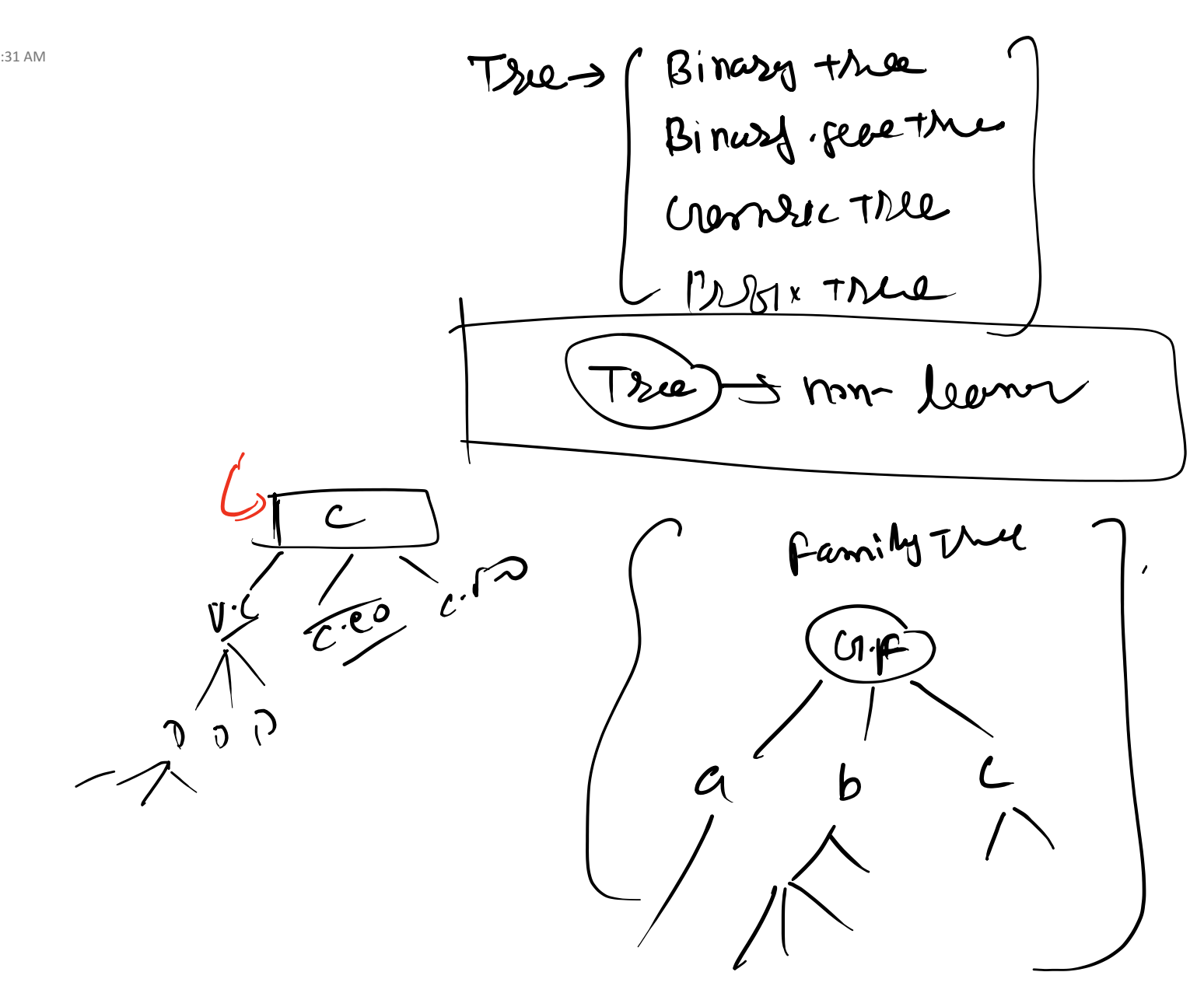
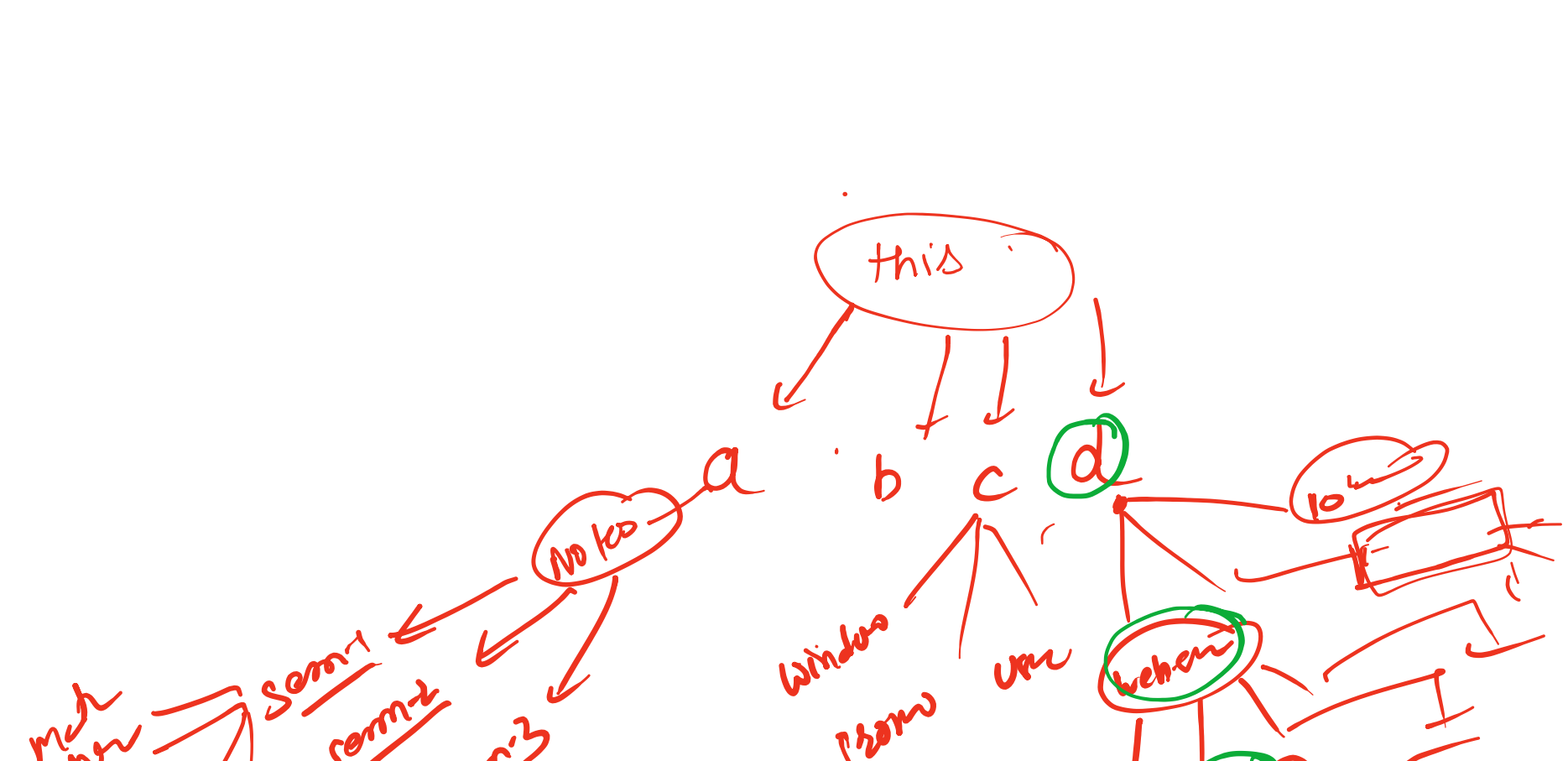
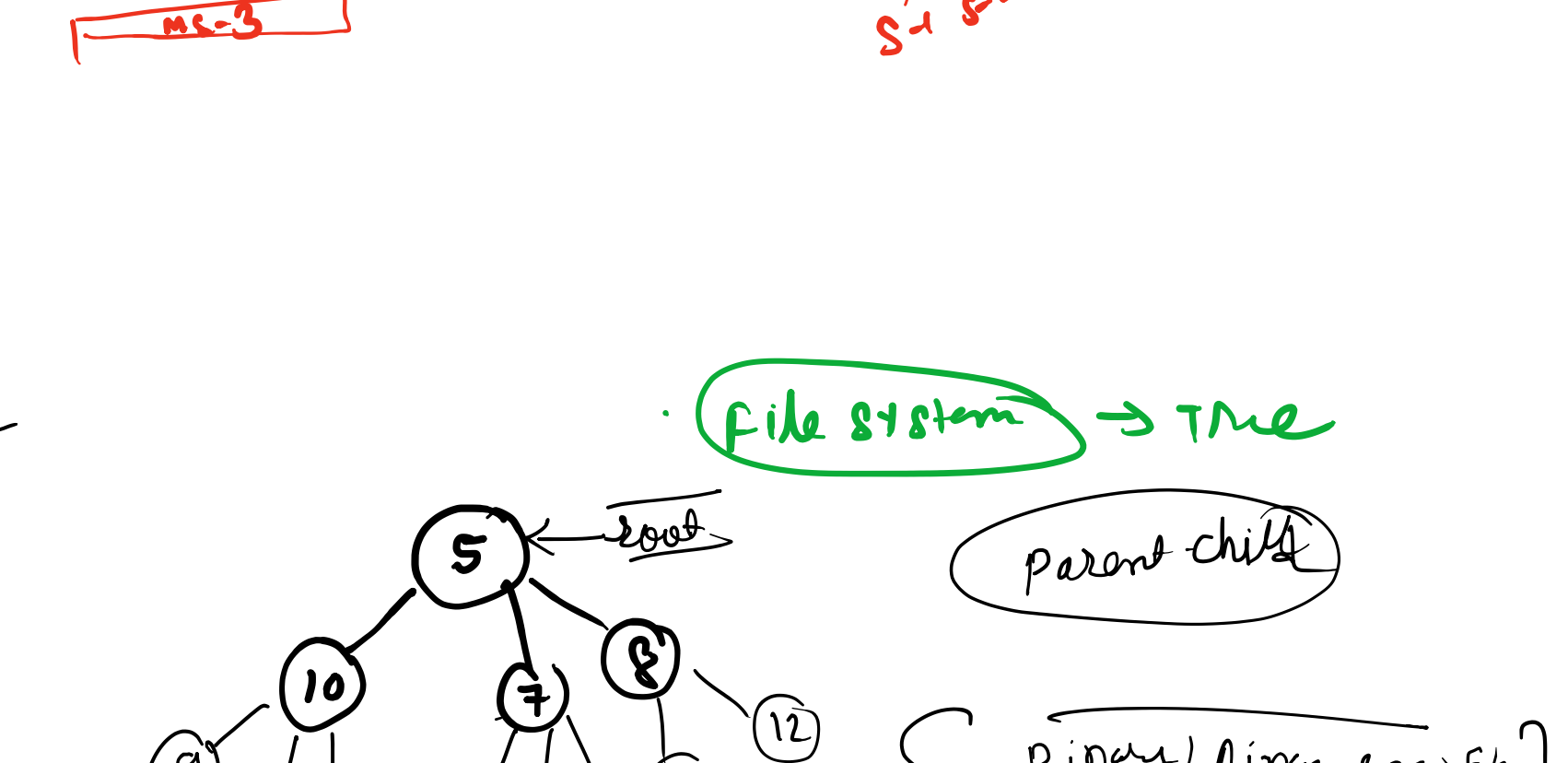
Lec-25

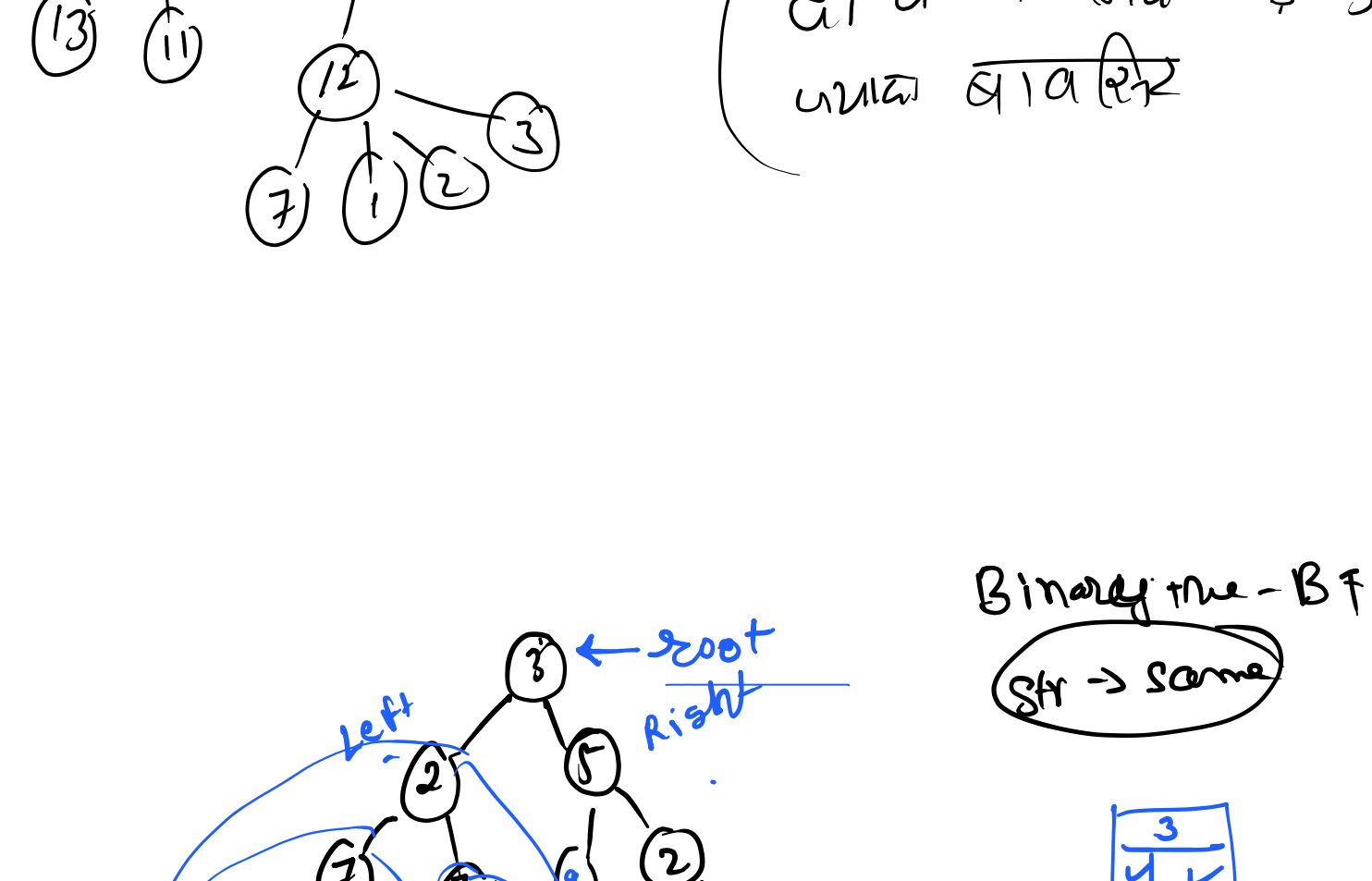
23/08/25 9:31 AM

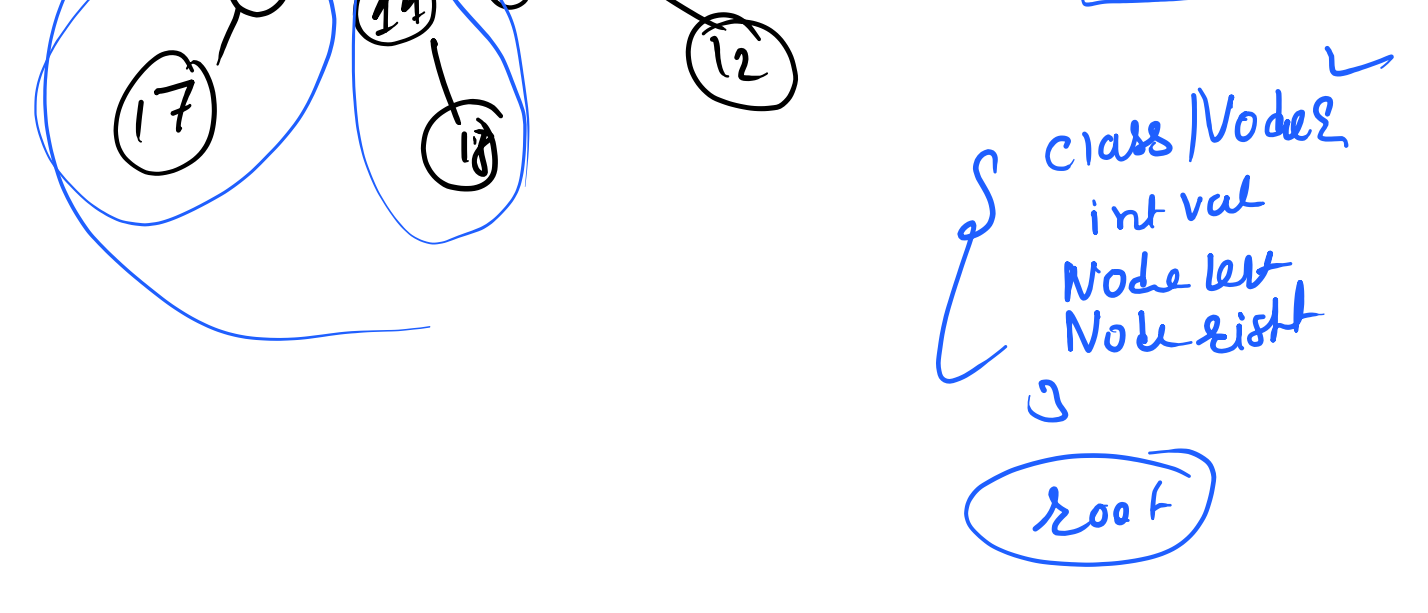

Tree →
Binary tree
Binary search tree
Generic tree
Prefix tree
Tree → non linear
C
V.C
C.E.O
C.T.O
Family Tree
G.F
a
b
c


this
a
b
c
d
No Ko
Sem1
Sem2
Sem3
windows
Iphone
web
PM
MR
MS-3

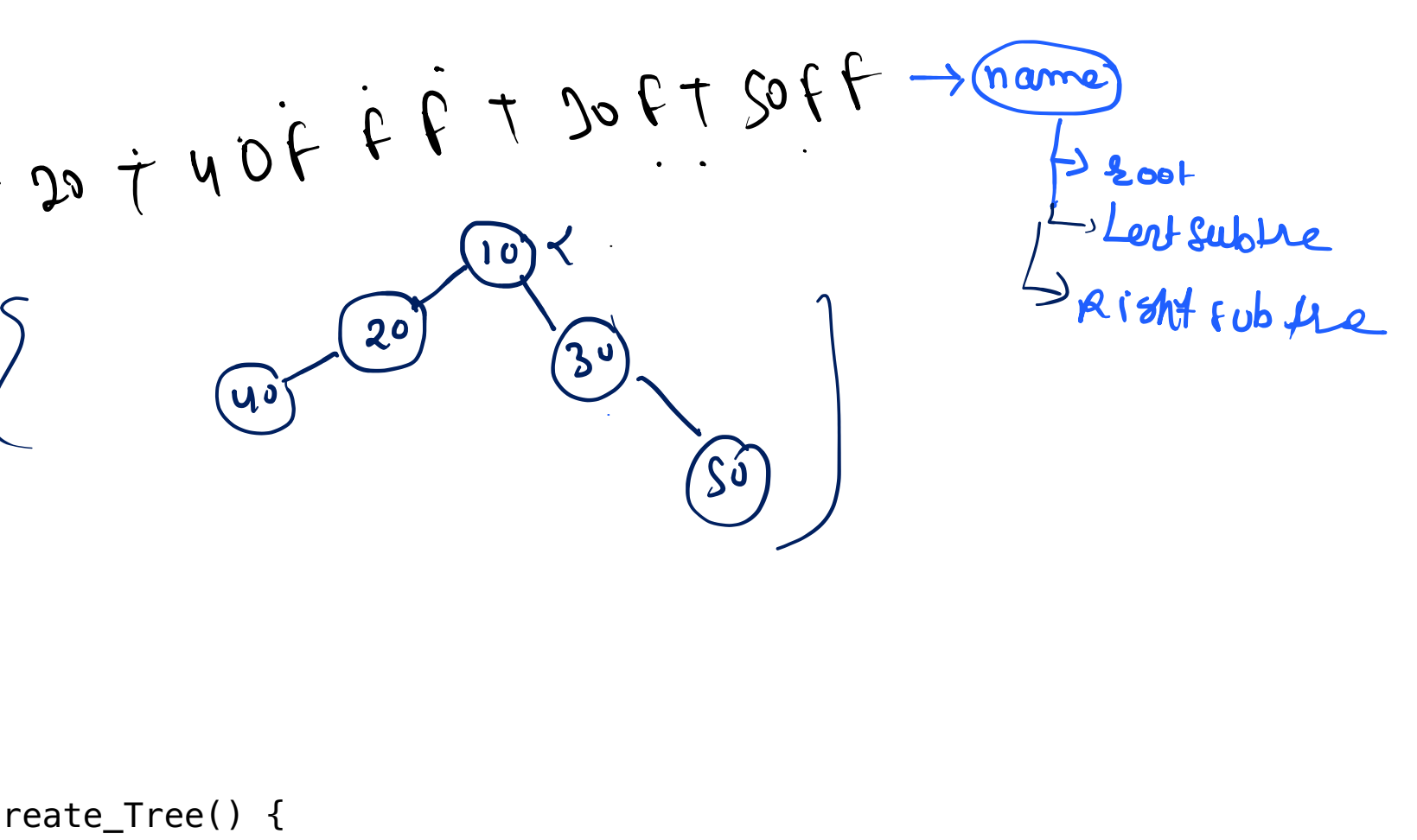
may


File system → Tree
root
Parent child
Binary / Binary search
5
10
7
8
9
7
2
11
7
6
1.5
12
13
11
12
3
7
1
2

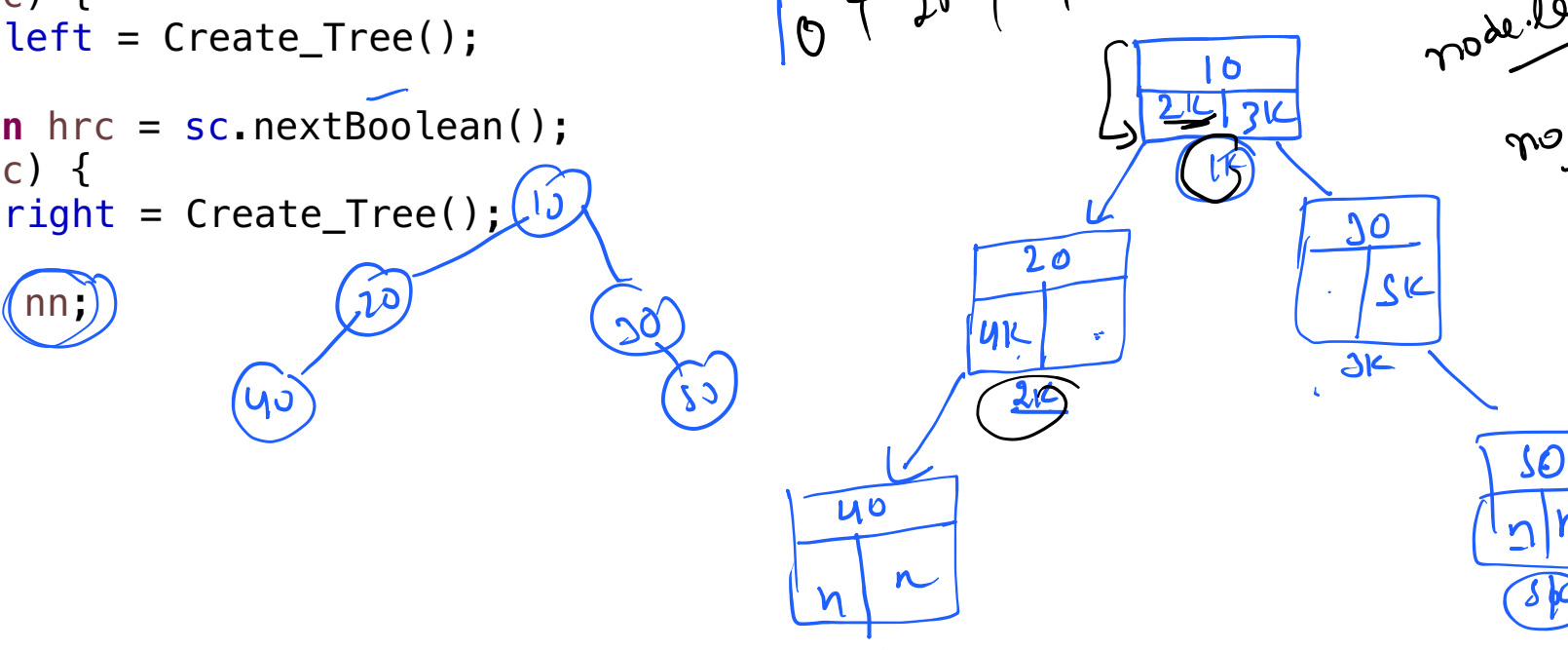

root
Left
Right
3
2
5
7
14
9
2
17
18
12
Binary tree - BT
str → same
class Node{
int val
Node left
Node right
}
root


10 T 20 T 40 F F F T 30 F T 50 F F
10
20
30
40
50
name
→ root
→ Left subtree
→ Right subtree

```java
private Node Create_Tree() {
		// TODO Auto-generated method stub
		int item = sc.nextInt();
		Node nn = new Node(item);
		boolean hlc = sc.nextBoolean();
		if (hlc) {
			nn.left = Create_Tree();
		}
		boolean hrc = sc.nextBoolean();
		if (hrc) {
			nn.right = Create_Tree();
		}
		return nn;
	}
```


10 T 20 T 40 F F F T 30 F T 50 F F
node.left != null
node.left.val
10
2k
3k
20
4k
40
n
n
4k
30
5k
3k
50
n
n
5k

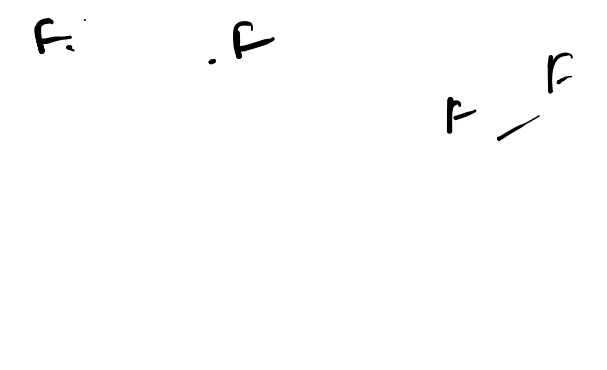
```java
String s="<--"+node.val+"-->";
```


20 ←10→
30
10
20
30
40
50
20 ← 10 → 30
40 ← 20 → .
. ← 40 → .
. ← 30 → 50
. ← 50 → .

```java
public void Display(Node node) {
		String s="<--"+node.val+"-->";
		if(node.left!=null) {
			s=node.left.val+s;
		}
		else {
			s="."+s;
		}
		if(node.right!=null) {
			s=s+node.right.val;
		}
		else {
			s=s+".";
		}
		System.out.println(s);
		Display(node.left);
		Display(node.right);
	}
```

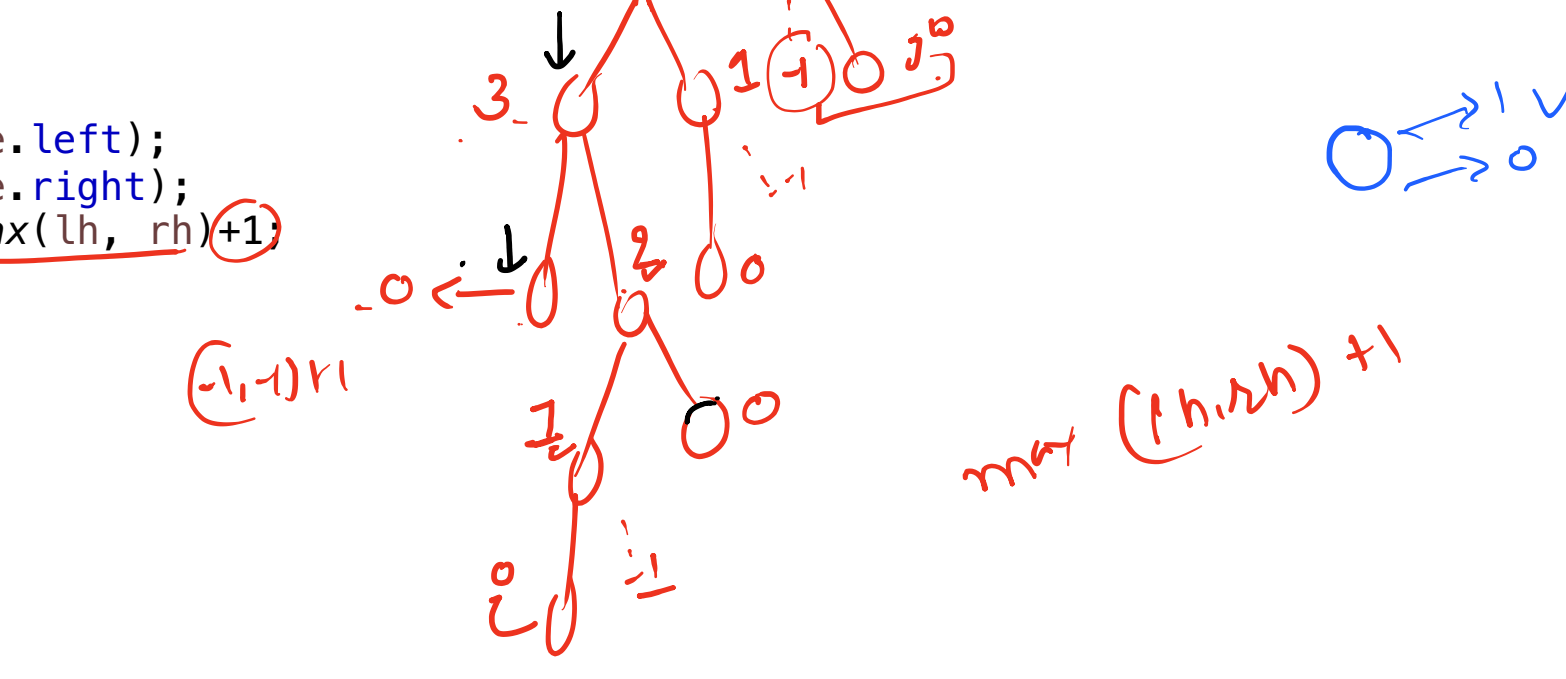
nw
20 ← 10 → 30 =
40 ← 20 → .
. ← 40 → .
. ← 30 → 50
. ← 50 → .
10
20
30
40
50
null

```java
private boolean find(Node nn, int item) {
		if (nn == null) {
			return false;
		}
		if(nn.val==item) {
			return true;
		}
		boolean left=find(nn.left, item);
		boolean right=find(nn.right, item);
		return left|| right;
	}
```


3
2
5
7
11
9
2
17
18
12
F
F
F
F
F
F
F
F

nn → -∞

```java
private int ht(Node node) {
		if(node==null) {
			return -1;
		}

		int lh=ht(node.left);
		int rh=ht(node.right);
		return Math.max(lh, rh)+1;
	}
```


5
=4
3
1
-1
0
-1
(-1,-1)+1
max (lh,rh) +1

maximum distance root to leaf node

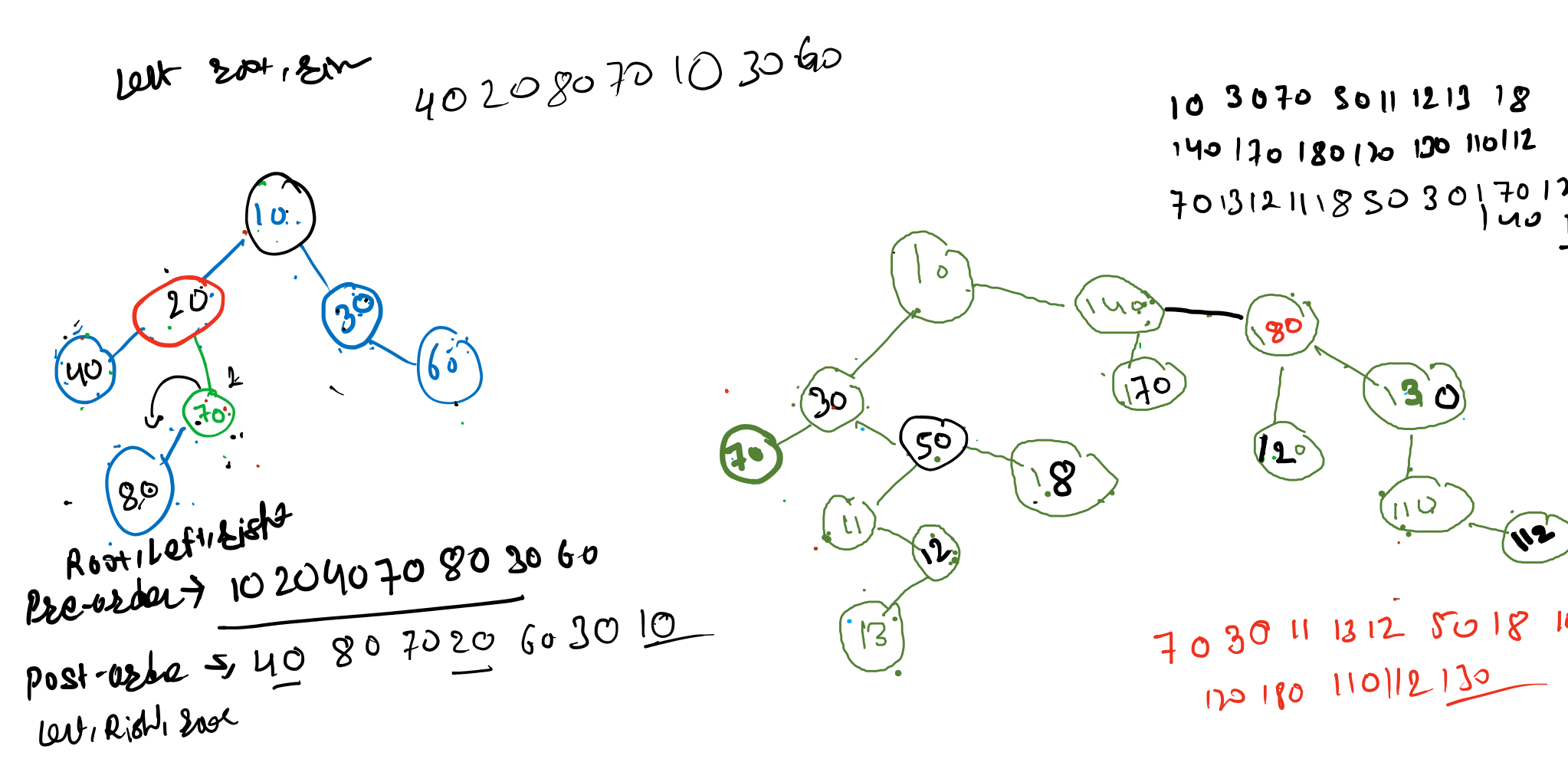
(i) Pre-order → Root, Left, Right

(ii) Post-order → Left, Right, Root

(iii) In-order → Left, Root, Right

(iv) Level-order

Left Root Right 40 20 80 70 10 30 60


10
20
30
40
70
60
80
10
30
140
70
50
18
11
12
13
80
120
130
110
112

Root Left Right

Pre-order → 10 20 40 70 80 30 60

Post-order → 40 80 70 20 60 30 10

Left, Right, Root

10 30 70 50 11 12 13 18

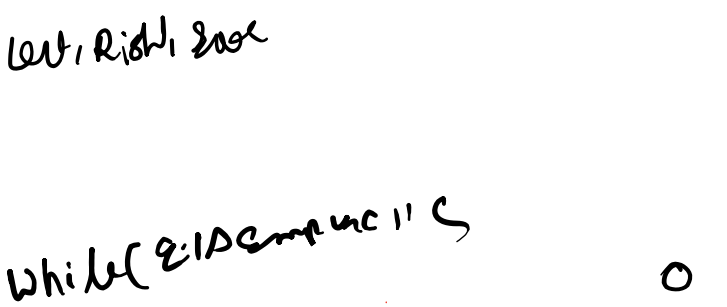
140 120 180 170 150 110 112

70 13 12 11 18 50 30 | 70 120 112 110 130 140 10

70 30 11 13 12 50 18 10 170 140

120 180 110 112 130

while(size != 0)


0
1
2
3
4
5
6
10
20
30
40
60
70
80

0 1 2 3 4 5 6

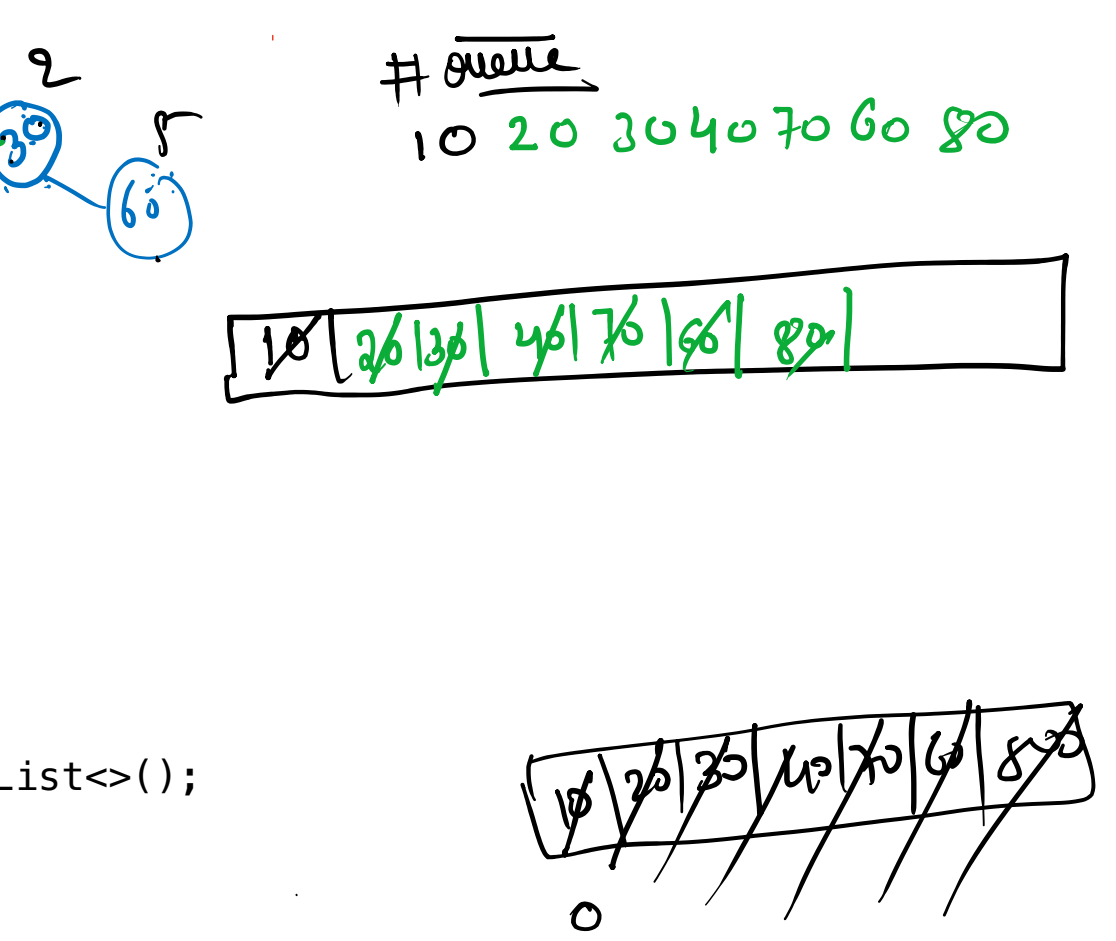
10 20 30 40 70 60 80

Queue

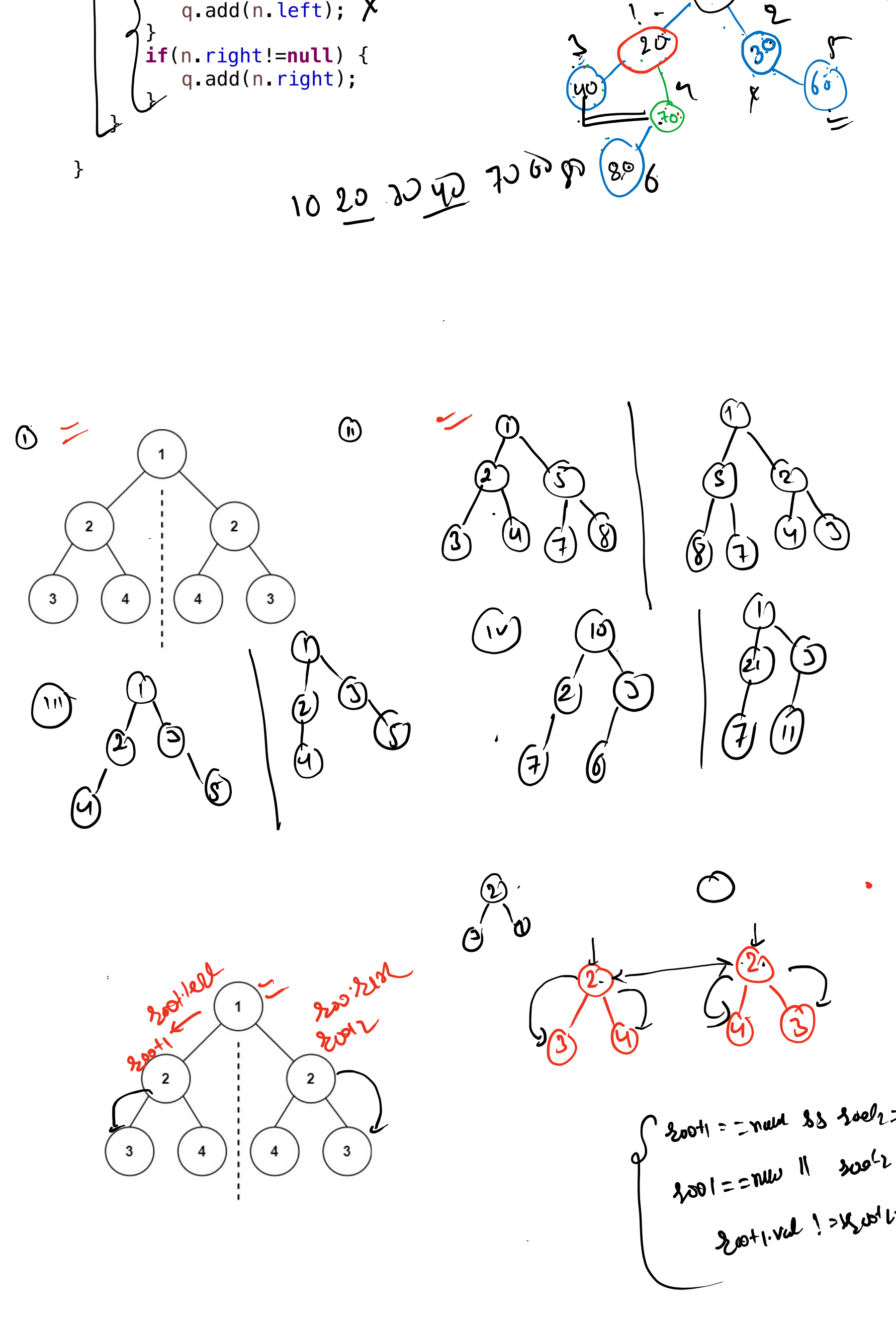
10 20 30 40 70 60 80

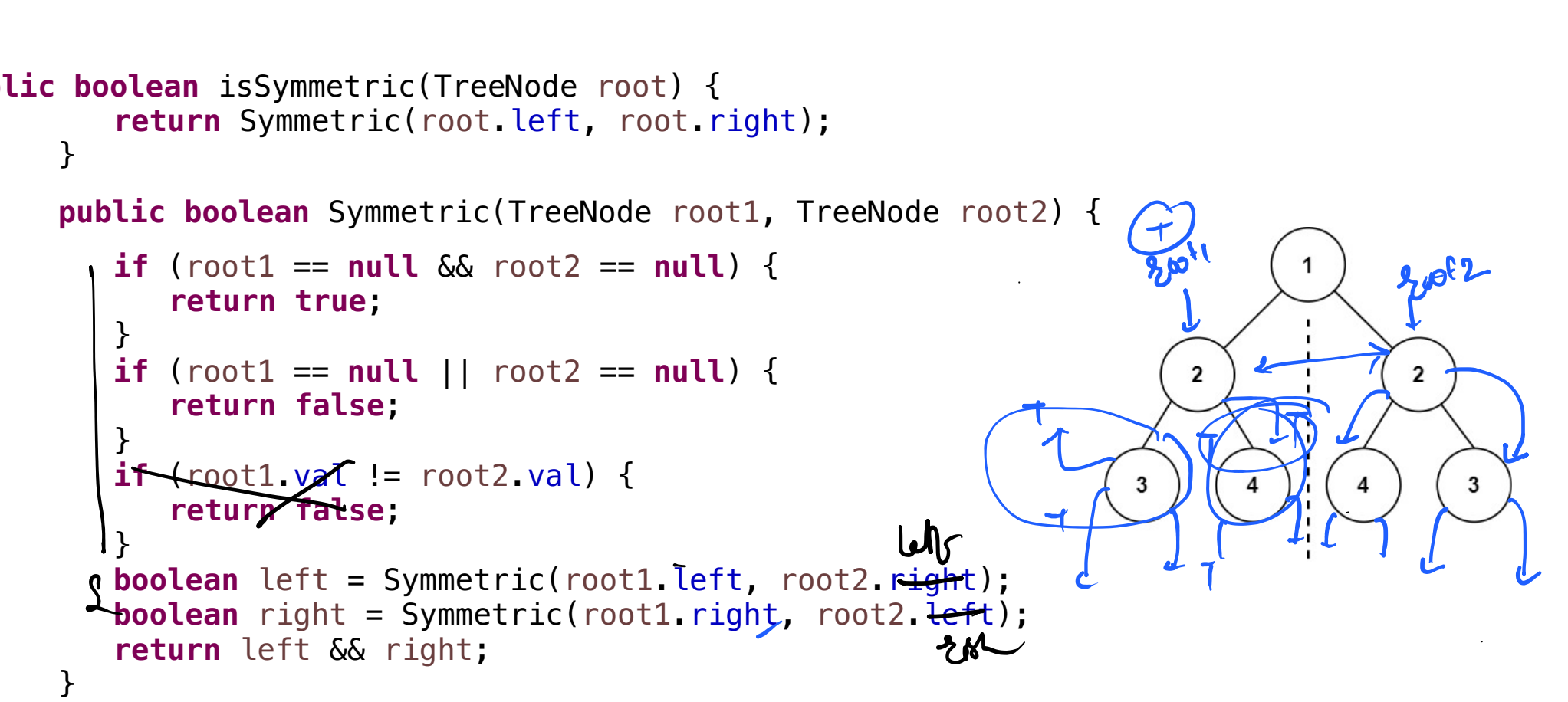
10	20	30	40	70	60	80

```java
public void LevelOrder() {
		Queue<Node> q = new LinkedList<>();
		q.add(root);
		while(!q.isEmpty()) {
			Node n=q.poll();
			System.out.println(n.val+" ");
			if(n.left!=null) {
				q.add(n.left);
			}
			if(n.right!=null) {
				q.add(n.right);
			}
		}
	}
```


10
20
30
40
70
60
80
10
20
30
40
70
60
80

10 20 30 40 70 60 80


(i)
1
2
2
3
4
4
3
(ii)
1
2
3
3
4
7
8
1
3
2
8
7
4
3
(iii)
1
2
3
4
5
1
2
3
4
5
(iv)
10
2
3
7
6
1
2
3
7
11

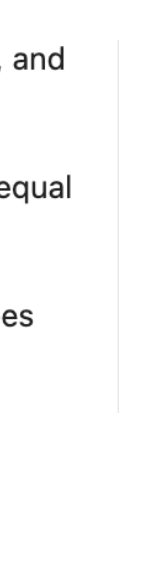

root1.left
root1
root2.right
root2
1
2
2
3
4
4
3
2
3
4

root1 == null && root2 == null ✓

root1 == null || root2 == null X

root1.val != root2.val X

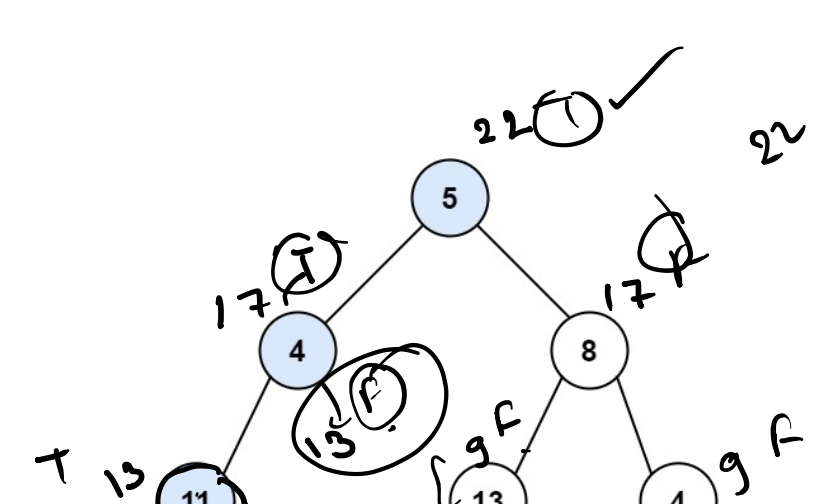
```java
public boolean isSymmetric(TreeNode root) {
		return Symmetric(root.left, root.right);
	}

	public boolean Symmetric(TreeNode root1, TreeNode root2) {
		if (root1 == null && root2 == null) {
			return true;
		}
		if (root1 == null || root2 == null) {
			return false;
		}
		if (root1.val != root2.val) {
			return false;
		}
		boolean left = Symmetric(root1.left, root2.right);
		boolean right = Symmetric(root1.right, root2.left);
		return left && right;
	}
```

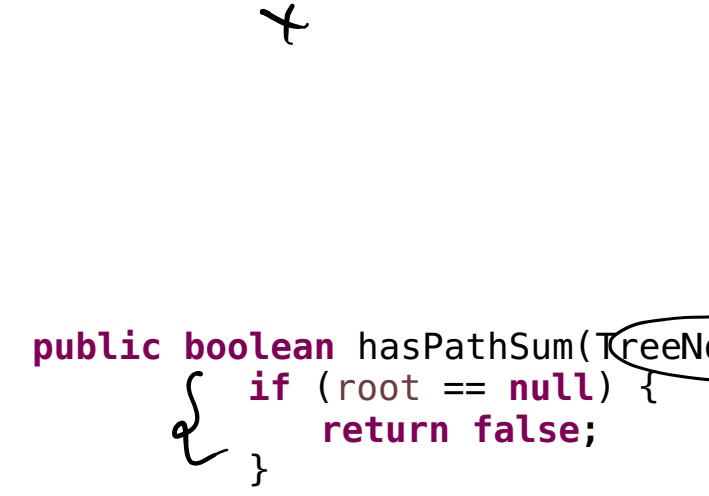

T
root1
root2
1
2
2
3
4
4
3
left
right

For a binary tree **T**, we can define a **flip operation** as follows: choose any node, and swap the left and right child subtrees.

A binary tree **X** is *flip equivalent* to a binary tree **Y** if and only if we can make **X** equal to **Y** after some number of flip operations.

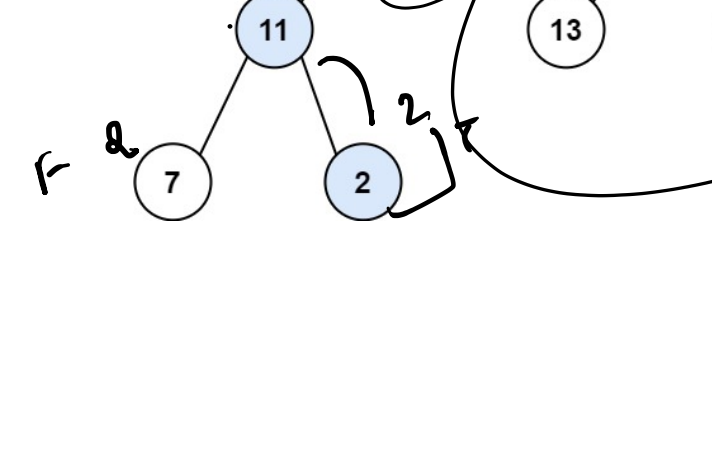
Given the roots of two binary trees `root1` and `root2`, return `true` if the two trees are flip equivalent or `false` otherwise.

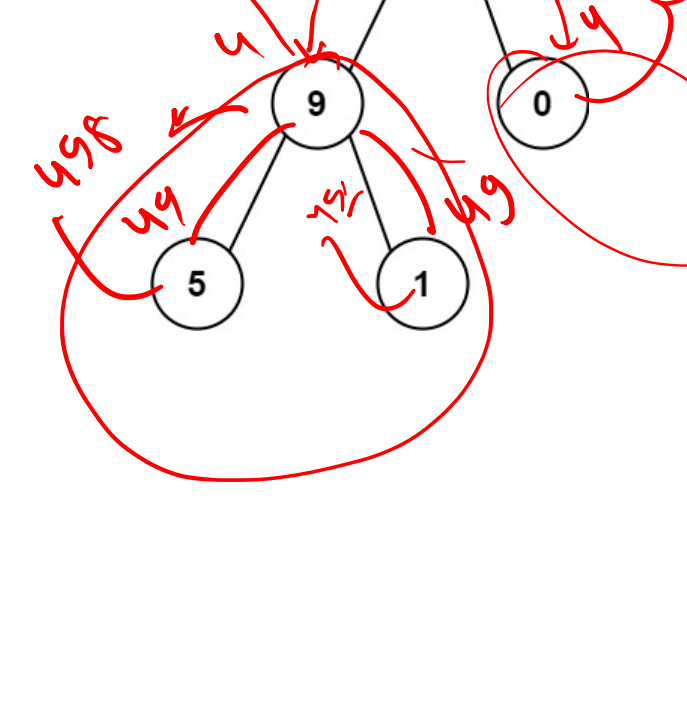

1
2
3
4
5
6
7
8
1
3
2
6
4
5
8
7


22
5
17
4
T
8
17
F
11
13
T
13
F
13
9 F
4
9 F
7
2
2
T
OX
1
5 F
F

root, leaf → path target

```java
public boolean hasPathSum(TreeNode root, int targetSum) {
		if (root == null) {
			return false;
		}
		if (root.left == null && root.right == null) {
			return targetSum - root.val==0;
		}
		boolean left = hasPathSum(root.left, targetSum - root.val);
		boolean right = hasPathSum(root.right, targetSum - root.val);
		return left || right;
	}
```


5
17
4
T
8
17
F
13
11
13
13
4
7
2
2
1

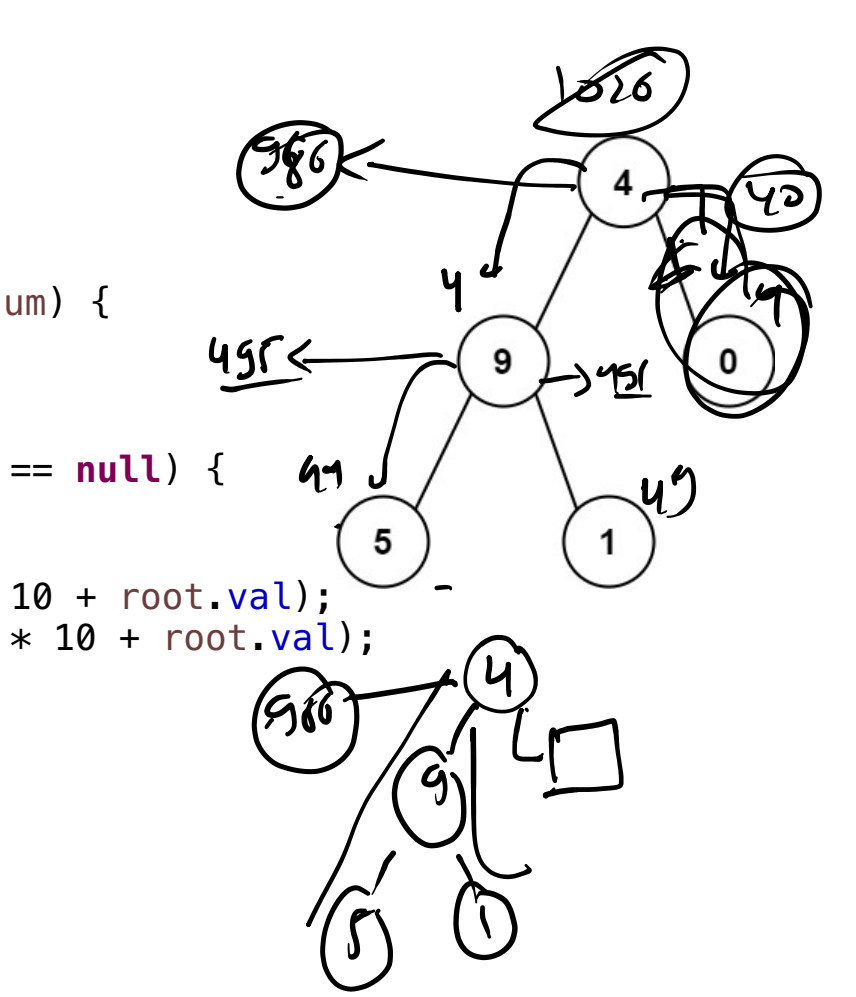

986
4
49
9
0
40
495
491
5
1

495 + 491 + 40 = 1026

495 + 491 = 986

986
4
49
9
0
495
491
5
1

```java
public int sumNumbers(TreeNode root) {
		return Numbers(root, 0);
	}

	public int Numbers(TreeNode root, int sum) {
		if (root == null) {
			return 0;
		}
		if (root.left == null && root.right == null) {
			return sum * 10 + root.val;
		}
		int left = Numbers(root.left, sum * 10 + root.val);
		int right = Numbers(root.right, sum * 10 + root.val);
		return left + right;
	}
```


1026
986
4
40
495
9
49
0
5
1
4
9
5
1